

Servicios de temporización

- Necesidades de temporización
- Temporización en ADA
- Temporización en POSIX 1003.1a
- Temporización en POSIX 1003.1b
 - Reloj de referencia
 - Creación, programación y destrucción de temporizadores
 - Otras llamadas de utilidad

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

1

Servicios de temporización

- Tarea típica:
 - Hacer siempre
 - Esperar momento apropiado
 - Realizar operaciones
 - Fin hacer
- Según causa del desbloqueo:
 - Tareas activadas por eventos (“event driven”)
 - Tareas activadas por tiempo
- Soluciones para tareas activadas por eventos:
 - La más elemental: Muestreo (“polling”)
 - Respuesta a interrupciones

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

2

Servicios de temporización

- **Servicios de temporización requeridos:**
 - Acciones periódicas: “Avísame cada 30 ms”
 - Aviso aislado
 - Retraso: “Espera 100 ms”
 - Sobretiempo: “Espera hasta que termine esta operación durante T ms como máximo”
- **Diferentes referencias**
 - Relativa: “Espera 1 ms” (desde ahora...)
 - Absoluta: “Espera hasta las 13:30”
- **Soluciones (tarea activada por tiempo):**
 - La más elemental: Temporización por programa (“ballast coding”)
 - Reloj de tiempo real y temporizadores

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

3

Temporización por programa

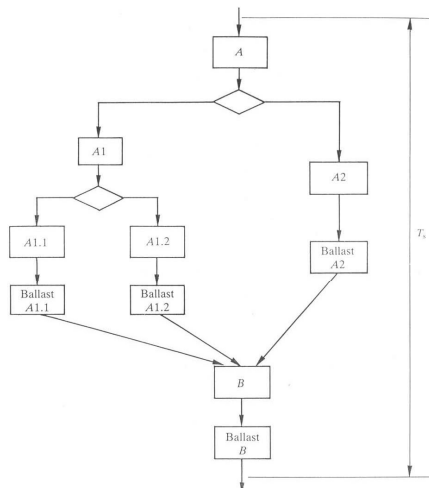


Fig. 4.2 Ballast coding.

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

4

Servicios de temporización

- Reloj de tiempo real y temporizadores
 - Reloj de tiempo real:
 - Referencia de tiempos (hora y calendario)
 - Permite medir intervalos
 - Permite el funcionamiento de temporizadores
 - Temporizadores: Generan eventos de temporización
 - Repetitivos
 - Disparo único, con referencia absoluta o relativa
 - Los eventos actúan sobre el programa:
 - Desbloquear tarea (acción periódica, retraso...)
 - Interrumpir acciones en curso (sobretiempo)
 - Activar acción asíncrona

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

5

Temporización periódica basada en esperas

- ¿Esperas relativas para temporización periódica?
- Objetivo: Ciclo T, Tan preciso como el reloj de referencia

- Primer intento:

Hacer Siempre
Control
Esperar(T)
Fin hacer

- El ciclo es mayor que T
 - Espera mayor que T
 - Coste del resto del ciclo no nulo

- Segundo intento:

Hacer Siempre
Control
Esperar(T-D)
Fin hacer

- Corrección fija (D)
- Hay errores:
 - Coste del resto del ciclo no constante
 - Otras perturbaciones

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

6

Temporización periódica basada en esperas

- Tercer intento:

Hacer Siempre

$T1 = \text{reloj}()$

Control

$T2 = \text{reloj}()$

$D = T2 - T1$

Esperar($T-D$)

Fin hacer

- Corrección variable

- Más precisa
- Pero no siempre podemos medir la perturbación

- Errores locales:

- Granularidad del reloj
- Latencia de interrupciones
- Expulsión por tareas más prioritarias

- Los errores locales son **acumulativos**

- No sucede con

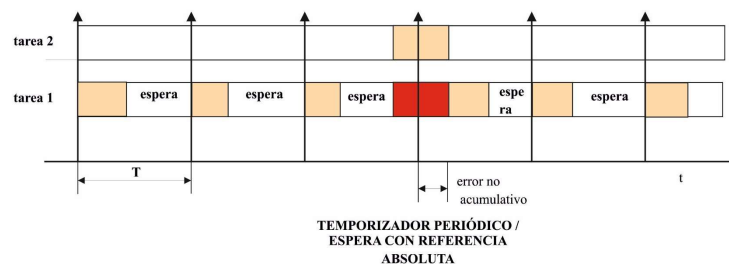
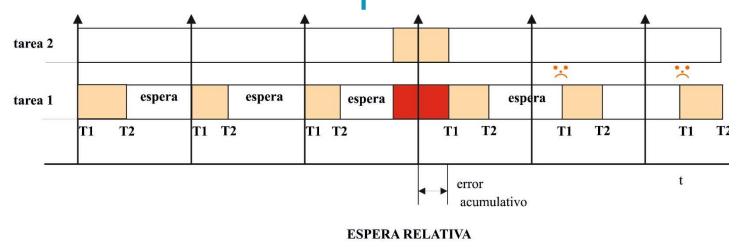
- Temporizadores periódicos
- Temporizadores de disparo único con referencia **absoluta**

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

7

Temporización periódica basada en esperas



02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

8

Temporización en ADA

- Package **Calendar** (obligatorio):
 - **Time**: Tipo que almacena un instante de tiempo.
 - **Duration**: Tipo que almacena un resultado de cálculos de tiempo
 - Número real en **punto fijo**. Indica segundos.
 - **Granularidad** de 20 ms como máximo
 - **Extensión** de -86400.0 a 86400.0 como mínimo
 - La resta de dos **Time** puede generar un resultado **Duration**
 - **Clock**: función que permite tener acceso al tiempo actual
- “Package” **Real_time**: opcional, con mayor resolución del reloj

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

9

Temporización en ADA

- Intervalo de tiempo invertido en un conjunto de sentencias:


```

declare
  t1, t2: Time;
  intervalo: Duration;
begin
  t1 := Clock;
  <más sentencias>
  t2 := Clock;
  intervalo := t2 - t1;
end;
      
```
- Retraso de tiempo **relativo** (sentencia **delay**):


```

      <sentencias1>
      delay 10.0;
      <sentencias2>
      
```
- Retraso de tiempo **absoluto** (sentencia **delay until**):


```

      Comienzo := Clock;
      <Sentencias1>
      delay until Comienzo + 10.0;
      <Sentencias2>
      
```

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

10

Temporización en ADA

- Temporización periódica:

- Los retrasos **delay** introducen incertidumbres y deriva **acumulativa**.
- Esto no sucede con una referencia absoluta: **delay until**

```

declare
  Proximo: Time;
  Intervalo: constant Duration := 7.0;
begin
  Proximo := Clock + Intervalo;
  loop
    <sentencias>
    delay until Proximo;
    Proximo := Proximo + Intervalo;
  end loop;
end;

```

- Sobretiempo con la sentencia **select**:

```

select
  accept entrada1(l: Integer) do
    <sentencias>
  end entrada;
or
  delay 5.0;
  <sentencias alternativas>
end select;

```

- Sobretiempo para abortar una actividad ya comenzada:

```

select
  delay 0.5;
  <sentencias A>
then abort
  <sentencias B>
end select;

```

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

11

Temporización en POSIX 1003.1a

- Reloj de referencia:

```
time_t time(time_t *tiempo);
```

- Retraso relativo y temporizador:

```

unsigned long sleep(unsigned segs);
unsigned alarm(unsigned segs);

```

- Problemas:

- Poca resolución (segundos)
- No hay temporizadores periódicos ni con referencia absoluta
- **alarm** y **sleep** pueden interaccionar (si **sleep** usa SIGALRM; pero nunca lo hará en entorno multihilo)

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

12

Temporización en POSIX 1003.1b

- **Mejoras:**
 - Mayor resolución (hasta nanosegundos)
 - Temporizadores con diversos modos
 - Asignación flexible de señales a eventos
 - Detección de “overruns”
- **Reloj de referencia (CLOCK_REALTIME):**
 - Obtener fecha/hora y resolución:


```
int clock_gettime(clockid_t reloj, struct timespec *tiempo);
int clock_getres(clockid_t reloj, struct timespec *resol);
```
- **Retraso relativo con mayor resolución:**

```
int nanosleep(struct timespec *retraso, struct timespec *queda);
```

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

13

Temporización en POSIX 1003.1b

- **Temporizadores:**
 - Creación: Hay que especificar el reloj de referencia y el modo de notificación del evento


```
int timer_create(clockid_t reloj, struct sigevent *aviso,
                  timer_t *tempo);
```
 - Programación:
 - Dos valores: **it_value** e **it_interval**, en **struct itimerspec**
 - Flag **TIMER_ABSTIME**

```
int timer_settime(timer_t tempo, int flags,
                  const struct itimerspec *spec,
                  struct itimerspec *spec_ant);
```

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

14

Temporización en POSIX 1003.1b

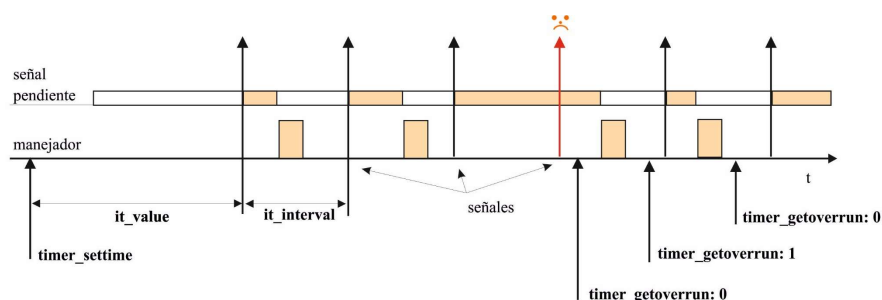
- Programación de temporizadores:
 - Si **it_value** = 0:
 - Temporizador **desactivado**
 - Si **it_value** != 0:
 - **it_interval** = 0: Temporizador de **un solo disparo**
 - **it_interval** != 0: Temporizador **periódico**
 - Significado:
 - **it_value**: Define el instante del primer evento
 - **it_interval**: Define el periodo de activación
 - Flag **TIMER_ABSTIME**: **it_value** se interpreta como un instante de **tiempo absoluto**
- No es necesario destruir un temporizador para reprogramarlo
- **Nunca se encola** más de una activación de la señal de un temporizador a la vez

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

15

Programación de un temporizador



- **Nunca se encola** más de una activación de la señal de un temporizador a la vez
 - En un temporizador cíclico si la señal no se recibe **se pierde** cuando llegue la siguiente, con o sin **SA_SIGINFO** ("overrun")

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

16

Programación de un temporizador

```
#include <time.h>
#include <signal.h>

timer_t mitemp;
struct sigaction accion;
siginfo_t info;
struct sigevent eventos;
struct itimerspec prog;
struct timespec ciclo;
sigset_t ev_tempo;
.....
/* Programación: Primer disparo a los 1sg
   250mseg.; ciclo de 1 sg. 250 mseg. */
ciclo.tv_sec = 1;
ciclo.tv_nsec = 250000000L;
prog.it_value = ciclo;
prog.it_interval = ciclo;

/* SIGRTMIN de tiempo real */
accion.sa_sigaction = manejador;
accion.sa_flags = SA_SIGINFO;
sigemptyset(&accion.sa_mask);
sigaction(SIGRTMIN, &accion, NULL);

/* El temporizador avisará con una señal SIGRTMIN */
eventos.sigev_signo = SIGRTMIN;
eventos.sigev_notify = SIGEV_SIGNAL;
eventos.sigev_value.sival_ptr = (void *)&mitemp;

/* Bloqueo de SIGRTMIN para usar sigwaitinfo */
sigemptyset(&ev_tempo);
sigaddset(&ev_tempo, SIGRTMIN);
sigprocmask(SIG_BLOCK, &ev_tempo, NULL);

/* Temporizador creado y activado */
timer_create(CLOCK_REALTIME, &eventos, &mitemp);
timer_settime(mitemp, 0, &prog, NULL);

(...)
/* Esperando señal (vencimiento del temporizador) */
sigwaitinfo(&ev_tempo, &info);
(...)

/*Destrucción del temporizador */
timer_delete(mitemp);
....
```

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

17

Otras llamadas útiles

- **Lectura del estado del temporizador:**
 - **it_value:** Tiempo hasta próxima activación
 - **it_interval:** Intervalo real programado

```
int timer_gettime(timer_t tempo, struct itimerspec *queda);
```
- **Lectura de la cuenta de “overrun”:**
 - Número de señales generadas (y “perdidas”) desde que quedó pendiente la última recibida:

```
int timer_getoverrun(timer_t tempo);
```
- **Convertir fecha en tiempo local a **time_t**:**

```
time_t mktime(struct tm *fecha);
```

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

18

Otras llamadas útiles

- Obtener fecha en tiempo local:
 - **tiempo**: Tiempo en segundos (obtenido con **time**, por ejemplo)


```
struct tm *localtime(time_t *tiempo);
```
- Obtener fecha en tiempo de referencia (UTC, Greenwich)
 - Funciona como **localtime**:


```
struct tm *gmtime(time_t *tiempo);
```
- Obtener fecha como cadena de caracteres:
 - **fecha**: Fecha en formato numérico (obtenida con **localtime** o **gmtime**, por ejemplo)


```
char *asctime (struct tm *fecha);
```

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

19

Temporizador con referencia absoluta

```
#include <time.h>
#include <signal.h>

timer_t mitemp;
siginfo_t info;
struct itimerspec prog;
sigset_t ev_tempo;
struct tm fecha;
struct timespec cero = {0,0};
.....
(crear temporizador con timer_create; preparar
tratamiento de señal y conjunto de señales)
.....
/* Programación: Avisar a las 11:50:27 del
día 10/11/24, hora local */

fecha.tm_hour = 11;
fecha.tm_min = 50;
fecha.tm_sec = 27;
fecha.tm_mday = 10;
fecha.tm_mon = 11-1; /* Comienza en 0... */
fecha.tv_year = 124; /* 0 es 1900 */

/* Con mktime se obtiene la cuenta de segundos */
prog.it_value.tv_sec = mktime(fecha);
prog.it_value.tv_nsec = 0; /* Podría no ser 0... */

/* Disparo único (pero podría ser periódico si it_interval
no fuese cero) */
prog.it_interval = cero;

/* Programar temporizador usando TIMER_ABSTIME */
/* El it_value se toma como tiempo absoluto */
timer_settime(mitemp, TIMER_ABSTIME, &prog, NULL);

(...)
/* Esperar señal del temporizador */
sigwaitinfo(&ev_tempo, &info);
(...)
```

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

20

Temporizadores: Puntos a recordar

- Para medir intervalos:
 - No hacen falta; basta con el reloj de tiempo real (`clock_gettime`)
- Crear temporizador (`timer_create`):
 - Definimos la señal que emite y su dato. Ninguno de ellos puede cambiar
 - La señal va dirigida al proceso que lo crea, no a un hilo en particular
 - Varios temporizadores:
 - En principio podrían usar la misma señal con diferente dato
 - Pero si deben afectar a hilos distintos, normalmente no puede usarse la misma señal ¡no sabríamos cual la recibe!
- Tratar la señal:
 - No se encolan, en ningún caso; si no se tratan/reciben, se pierden
 - Podemos esperar síncronamente la señal en un hilo
 - Con un manejador podemos generar sobretiempo de esperas
 - Programar un manejador (puede bastar uno vacío...)
 - Bloquear señal en todos los hilos menos el que espera

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

21

Temporizadores: Puntos a recordar

- Programar temporizador (`timer_settime`):
 - Parámetros:
 - `it_value`: Tiempo hasta primera activación – si cero, desactivado
 - `it_interval`: Intervalo de repetición – si cero, un solo disparo
 - Tiempo absoluto (flag `TIMER_ABSTIME`) – `it_value` es tiempo absoluto
 - Cualquier hilo puede usar el temporizador, y no hace falta exclusión mutua (`mutex`; ya veremos qué es eso...)
 - No hace falta destruir y crear de nuevo un temporizador para reprogramarlo
 - Al reprogramar, la programación anterior queda anulada
 - No es adecuado usar un temporizador de un solo disparo para generar comportamiento cíclico (debe ser periódico...)
 - No hace falta reprogramar un temporizador periódico en cada ciclo (de hecho si lo hacemos, lo hará menos preciso...)

02/11/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

22

Temporizadores: Puntos a recordar

- Estructuras **timespec** e **itimerspec**:
 - En las **itimerspec**, **it_value** e **it_interval** son **estructuras timespec**; no pueden inicializarse como si fueran enteros
 - Sumar tiempo a una estructura **timespec**: **tv_nsec** **no puede pasar de 1000 millones** (1 segundo) – por tanto, **acarreo** a **tv_sec**

t1: 3 sg y 200*1000000 ns
 t2: 1 sg y 850*1000000 ns
 t1+t2 **NO puede ser** 4sg y 1050*1000000 ns (valor incorrecto)
 t1+t2 debe ser **5 sg y 50*1000000 ns**
 - Restar estructuras **timespec**: No vale restar separadamente los **tv_sec** y los **tv_nsec** (otra vez acarreo – esta vez negativo):

t1-t2 **NO puede ser** 2 sg y -650*1000000 ns
 t1-t2 debe ser **1 sg y 350*1000000 ns**